

Pense-bête : Integrated Gradient (Gradient Intégré)

SCHMITT Hugo

April 9, 2026

Introduit par **Mukund Sundararajan**, **Ankur Taly**, **Qiqi Yan** le 13 juin 2017 dans l'article "[Axiomatic Attribution for Deep Network](#)", propose deux axiomes fondamentaux que les méthodes d'attributions devraient satisfaire, ce qui n'est pas le cas de la plupart des méthodes. Ces axiomes sont mise en œuvre pour cette nouvelle méthode d'attribution dénommé *Integrated Gradient*. Comme son nom l'indique, il s'agit d'une méthode d'attribution de la famille des gradients (gradient-based).

Axiomes

Les deux axiomes principaux :

Sensibilité : pour chaque entrée x et baseline x' qui diffèrent en une caractéristique et qui ont des prédictions différentes, alors la caractéristique qui diffèrent devraient se voir attribuer une attribution non-nulle. (*NB: Les gradients, les approches de rétro-propagation et de déconvolution ne respectent pas cet axiome de sensibilité.*) De plus, si la fonction implémenté par le réseau de neurones ne dépend pas mathématiquement d'une ou plusieurs variables, alors l'attribution à cette variable sera toujours zéro.

Invariance à l'implémentation : Deux modèles sont *fonctionnellement équivalent* si leurs sorties sont égales avec les mêmes entrées, malgré des implémentations différentes. Les méthodes d'attributions devraient satisfaire *l'invariance à l'implémentation*, c'est-à-dire que pour deux modèles fonctionnellement équivalent, leurs attributions seront toujours identiques.

Complétude : les attributions se complètent avec la différence entre la sortie du modèle avec l'entrée de base x et de la sortie du modèle avec la baseline x' .

Linéarité : ...

Préservation de la symétrie : ...

Gradient Intégré

Formules :

(toutes tirées de l'article)

$$\text{IntegratedGradients}_i(x) = (x_i - x'_i) \int_0^1 \frac{\partial F(x' + \alpha(x - x'))}{\partial x_i} d\alpha$$

Où,

x : entrée
 x' : baseline (référence)
 α : constante d'interpolation
 F : fonction du modèle
 i : dimension de l'entrée

Qui se calcule informatiquement par une d'une somme de Riemman :

$$\text{IntegratedGradients}_i^{\text{approx}}(x) = (x_i - x'_i) \times \sum_{k=1}^m \frac{\partial F(x' + \frac{k}{m} \times (x - x'))}{\partial x_i} \times \frac{1}{m}$$

Avec m le nombre de pas dans l'approximation de Riemann.

On peut retenir une forme plus simple (applicable à la presque totalité des réseaux de neurones, voir *Proposition 1 de l'article*)

$$\sum_{i=1}^m \text{IntegratedGradients}_i(x) = F(x) - F(x')$$

Plus simplement, on attribue à chaque entrée une importance en intégrant sa contribution (via le gradient) le long du chemin reliant la baseline à l'entrée. On accumule alors son effet sur la variation de la prédiction sur ce chemin. On peut dire qu'on regarde le dénivelé total parcouru depuis la baseline, au lieu de regarder seulement la pente sous un point.

Ça évite, entre autre, les problèmes de saturations du gradient (lorsqu'en faisant le gradient à un point on perd sa contribution car la fonction d'activation l'a aplatie), on gagne en stabilité. Dans le même temps, on obtient une contribution total (le long du chemin), au lieu d'une sensibilité locale.

Rappel que le gradient mesure la sensibilité locale, et non l'importance.